

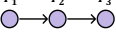
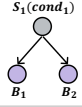
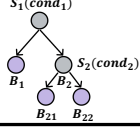
Composition Type	Description	Example	Illustration
Single	The output is required to satisfy a single constraint.	Please summarize the following news.	
And	The output is required to satisfy multiple constraints simultaneously.	Please summarize the following news. The summary should be output in bullet points , and within 100 words .	
Chain	The output is required to complete multiple tasks sequentially, each of which needs to satisfy its own constraints.	Please introduce "Mona Lisa" briefly. Firstly , introduce the year of creation, then describe the background of the work's creation, and finally , summarize the impact of the work.	
Selection	The output is required to select different branches according to certain conditions, fulfilling the constraints of the corresponding branch.	Please introduce the following painting. - If the work contains any animal , the description should be in English - Otherwise , the description should be in Chinese Painting: "Mona Lisa"	
Nested Structure	The above composition types are recursively nested to form more complex structures.	Analyse the sentiment of above user comment and complete the following tasks: 1. If it's positive : Identify the products within the comments ... 2. If it's negative , analyze the reasons for it: - If the reason is not about products itself , ... - Otherwise , ...	

Figure 3: Composition types in COMPLEXBENCH. Each node is a part of an instruction. The purple node may contain other composition types, while the blue node does not. In addition to 4 basic types, the last row also shows a nested selection type.

3.2 Constraint

Following existing works on controlled text generation and instruction following [31, 32, 33, 34, 19, 10], we propose a two-level structure for constraints including 4 constraint types (i.e., Lexical, Format, Semantic, and Utility) and 19 specific constraint dimensions which are further divided from the above types. The distribution of these constraint types and dimensions within COMPLEXBENCH is shown in Figure 2. We present the definitions of constraint types in the following and describe the details of the constraint dimensions in Appendix D.

Lexical Constraint requires to output specific keywords or phrases or precisely generate texts that are related to specific keywords mentioned in the instructions [35, 36, 34].

Format Constraint specifies the requirements on the output structure (such as JSON, Markdown, and bullet points), length, and patterns of the output, where the patterns include punctuation, content at the beginning or end, and the output templates. Format constraints require LLMs to possess a precise understanding and planning of the output content, which remain challenging for current LLMs [19, 23].

Semantic Constraint specifies the topic [37], language style [32], personality [10], and sentiment [38] of the output, which are common constraints in the existing works on controlled text generation.

Utility Constraint measures the language, helpfulness, supportiveness, consistency, and factuality of generated texts, which are holistic properties. Among these, helpfulness indicates whether the generated text can complete the basic task included in the instruction (such as *Please introduce the following painting.* in Figure 1) regardless of satisfaction of other constraints, while supportiveness means whether the generated text is faithful to the instruction.

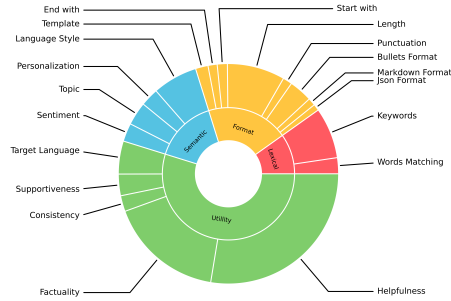


Figure 2: Constraint distribution of COMPLEXBENCH. The Utility constraints helpfulness and factuality possess a high proportion due to their prevalence in various instructions, which are basic requirements for high-quality outputs.

3.3 Composition

As shown in Figure 3, we propose 4 composition types that indicate typical structural combinations of constraints.

Single. The output is required to satisfy a single constraint, with no composition involved.

And. The output needs to satisfy multiple constraints simultaneously. This simple composition type commonly appears in most of the existing benchmarks on complex instruction-following [3, 19, 7].

Chain. The output is required to complete multiple tasks in the instruction sequentially, each of which may contain several constraints. Formally, *Chain* contains n tasks $\{T_1, T_2, \dots, T_n\}$, which need to be completed sequentially. The output of T_{k+1} may depends on that of T_k ($k = 1, 2, \dots, n - 1$).

Selection. The output is required to select different branches according to certain conditions, fulfilling the constraints of the corresponding branch. Formally, *Selection* contains m branches $\{B_1, B_2, \dots, B_m\}$, each of which is a task with expected outputs $Y_1, Y_2, \dots, Y_m$ respectively. We denote a selection function as S with a range $\{1, 2, \dots, m\}$, taking the selection condition *cond* as input. Finally, the expected output of the instruction is $Y_{S(cond)}$.

It’s worth noting that the above composition types can be nested to construct more complex structures. Each task in *Chain* and each branch in *Selection* may also contain other composition types. As shown in the last row of Figure 3, a branch of *Selection* can also contain *Selection*, thus forming a nested selection composition type.

To verify the necessity and comprehensiveness of the composition types considered in COMPLEXBENCH, we analyze the distribution of composition types in real-world scenarios. We collect instructions with high demand and representativeness from an online LLM-based chat service platform that serves more than a million users daily including general and professional instructions. General instructions refer to the instructions used by individual users in routine scenarios, while professional instructions refer to those used by enterprise-level users in business and research scenarios. For each category of instructions, we randomly sample 300 instructions and manually count the number of instructions containing each composition type. We found that the taxonomy of COMPLEXBENCH fully covers present composition types. As shown in Figure 4, although the composition types of general instructions are relatively simple and have already been covered by current benchmarks, professional instructions include more complex composition types, such as *Selection* and nested structures of multiple composition types, which have rarely been considered by current benchmarks. As LLMs have been gradually applied to deal with complex instructions in professional scenarios, it is necessary to evaluate their ability to follow instructions with multiple constraints composition.

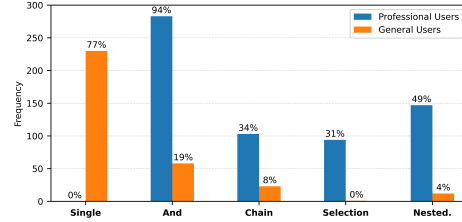


Figure 4: Composition type distribution of general and professional instructions.

4 COMPLEXBENCH Construction

4.1 Data Collection

We manually construct COMPLEXBENCH based on the taxonomy described in Section 3. The detailed construction pipeline consists of four steps, i.e., **Reference Instructions Collection**, **Task Allocation**, **Data Annotation and Validation**, and **Selection Branch Expansion**. We initially used our proposed method to construct Chinese data, while also providing an English version of COMPLEXBENCH. More details are in Appendix F, G, and H.

Reference Instruction Collection. Considering the difficulty of constructing complex instructions from scratch, annotators are required to create new complex instructions based on provided reference instructions. We collect reference instructions from real-world application scenarios and open-source instruction following benchmarks [19, 3, 7]. We conduct strict desensitization of privacy and carefully filter these instructions using category and quality classifiers.

Task Allocation. To ensure comprehensive coverage of each constraint and composition type, we partition the entire dataset construction into multiple annotation tasks. Each annotation task has different requirements for the minimal number of constraint dimensions in each constraint type and composition type. Annotators are required to modify reference instructions to meet the requirements of corresponding tasks. To alleviate the annotation cost, especially when the constraint

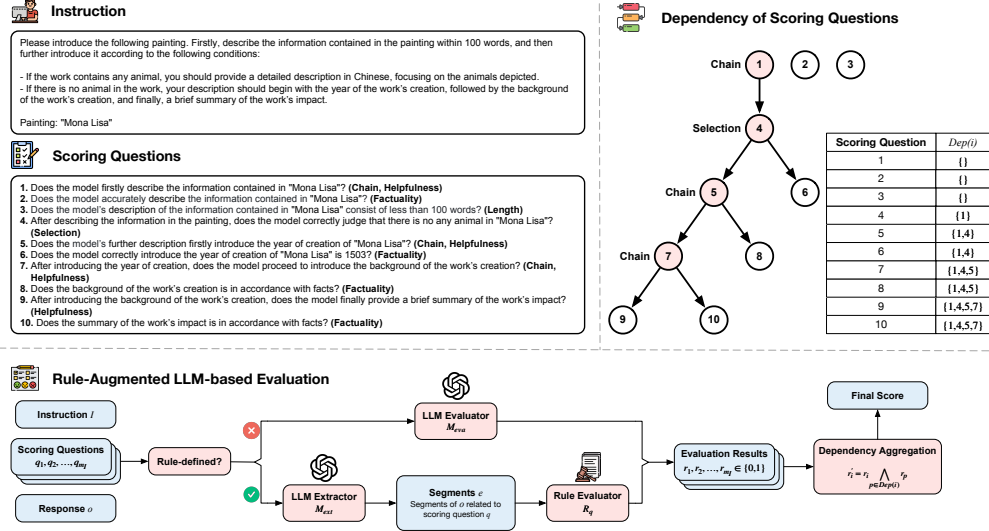


Figure 5: An exemplar evaluation process of COMPLEXBENCH. Given an instruction and its scoring questions, COMPLEXBENCH integrates the rule and LLM evaluator to verify each of them and aggregates the final score based on the dependency structure of composition types in the instruction.

dimensions in the reference instructions and task requirements are different, we leverage GPT-4 [39] to automatically acquire the constraint dimensions in reference instructions and assign them to corresponding annotation tasks according to minimal editing distance.

Data Annotation and Validation. Given reference instructions and corresponding annotation task requirements, annotators are expected to construct new complex instructions and annotate the constraint dimensions and composition types. After the data annotation, newly constructed instructions are cross-validated by other annotators. The process of validation continues until constructed instructions meet the following criteria: (1) Clarity & Reasonableness: The instruction should be easy to understand, unambiguous, and realistic, with at least one reasonable answer. (2) Validity of Constraints: Every constraint within the instruction should substantially influence the output. (3) Complexity & Difficulty: The instruction should be challenging for most LLMs and be capable of distinguishing the complex instruction-following abilities of different LLMs.

Selection Branch Expansion. When evaluating the ability of LLMs to follow instructions containing *Selection*, the predisposition toward random selection by LLMs may bring potential bias because most instructions cover only one correct selection branch. But the probability of the correct branch appearing at each position in *Selection* of data constructed by annotators is unequal². To address this issue, in the final stage of instruction construction, we manually modify the selection condition based on the selection function to construct multiple instructions that cover different correct branches, ensuring an equal probability of the correct branch appearing at each position.

4.2 Evaluation Protocol

To conduct a detailed evaluation of how well each constraint and composition type is satisfied, we draw inspiration from previous works that transform text evaluation into multiple question-answering tasks [16, 17, 7]. For each constraint and composition type specified in an instruction, we manually craft a scoring question that can be succinctly answered with either "yes" or "no."

Current mainstream evaluation methods contain LLM-based [18, 3, 7] and rule-based methods [8, 19, 20]. In our preliminary experiments, we find that LLM-based methods are effective at answering open-ended scoring questions, but they demonstrate a significant deficiency in those involving numerical computation, counting, and other objective rule-defined areas, such as keyword inclusion and text length. Simultaneously, rule-based methods perform well in rule-defined areas but are powerless against open-ended scoring problems. To address their limitations, we design a Rule-Augmented LLM-based (RAL) evaluation method to equip LLM evaluators with rules to answer

²For instance, after manual inspection we find that in all the selection compositions with two branches, annotators have about a 70% probability of selecting the first branch as the correct one.